

Trabajo Práctico N° 2

Entidades, Atributos, Ligaduras y Variables

Lectura: Sebesta Cap. 5, Scott Cap. 3

Conceptos

1. ¿A qué llamamos *entidad* en un programa? Brinde ejemplos de diferentes entidades que se pueden encontrar en un programa.
2. ¿Cuáles son los aspectos que deben tenerse en cuenta en la etapa de diseño de un lenguaje al especificar los atributos de los nombres? Analice ventajas y desventajas de las posibles decisiones para cada aspecto, incluyendo cómo pueden afectar a la legibilidad.
3. Sobre el concepto de *ligadura*:
 - a. ¿Qué es una ligadura? ¿Cómo se pueden clasificar los tiempos de ligadura? ¿A qué características o criterios de evaluación, en general, afecta el tiempo de una ligadura?
 - b. El concepto de ligadura, ¿está vinculado a la sintaxis o a la semántica de un lenguaje de programación? Justifique y ejemplifique.
 - c. ¿Cuál es la diferencia entre una ligadura implícita y una explícita? Brinde ejemplos y analice las ventajas y desventajas de cada una de ellas.
4. Describa cómo cada paradigma caracteriza a los atributos asociados a las variables.
5. ¿Cuáles son los aspectos que deben tenerse en cuenta en la etapa de diseño de un lenguaje al especificar los atributos de las constantes?
6. Presente ejemplos de conceptos que para su correcto funcionamiento necesiten ligarse estáticamente. Presente también ejemplos de conceptos que sea necesario ligarlos dinámicamente.
7. Defina las formas de asignación de memoria estática, basada en pila y heap. ¿Cuáles son las ventajas y desventajas de cada una de ellas?
8. ¿Con que tipo de asignación de memoria se relaciona la ligadura dinámica de tipo a una variable?
9. Considere un lenguaje con ligadura dinámica de tipos, y otro lenguaje con ligadura estática. Indique cómo esta característica puede afectar a la fiabilidad, flexibilidad y eficiencia en ejecución de los lenguajes.
10. Indique claramente las relaciones entre los conceptos de *visibilidad*, *alcance* y *ambiente de referenciamiento*.
11. Defina el concepto de *Aliasing*. ¿En qué situaciones puede ocurrir? ¿En qué ocasiones resulta útil? ¿Cuáles son sus desventajas?
12. ¿Qué entiende por reglas de alcance estático o dinámico? Analice las ventajas y desventajas de cada una de estas aproximaciones.
13. Considerando los siguientes programas, ¿qué valor de x es mostrado en el procedimiento *sub1*?

a. Bloques anidados - Alcance estático

```
program principal;
  var x: integer;

  procedure sub1;
    begin { sub1 }
      writeln('x =', x)
    end; { sub1 }

  procedure sub2;
    var x: integer;
    begin { sub2 }
      x := 10;
      sub1;
    end; { sub2 }

begin { principal }
  x := 5;
  sub2;
end; { principal }
```

b. Bloques chatos - Alcance dinámico

```
procedure sub1;
  begin { sub1 }
    writeln('x =', x)
  end; { sub1 }

procedure sub2;
  var x: integer;
  begin { sub2 }
    x := 10;
    sub1;
  end; { sub2 }

program principal
  var x: integer;
  begin { principal }
    x := 5;
    sub2;
  end; { principal }
```

14. Dadas las reglas de alcance estático, para cada uno de los siguientes programas, liste las variables que son visibles dentro de cada procedimiento.

```
program principal;
  var x, y, z: integer;

  procedure sub1;
    var a, y, z: integer;
    procedure sub2;
      var a, b, z: integer;
      begin { sub2 }
        ...
      end; { sub2 }
    begin { sub1 }
      ...
    end; { sub1 }

  procedure sub3;
    var a, x, w: integer;
    begin { sub3 }
      ...
    end; { sub3 }

begin { principal }
  ...
end; { principal }
```

```
program principal;
  var x, y, z: integer;

  procedure sub1;
    var a, y, z: integer;
    begin { sub1 }
      ...
    end; { sub1 }

  procedure sub2;
    var a, x, w: integer;
    procedure sub3;
      var a, b, z: integer;
      begin { sub3 }
        ...
      end; { sub3 }
    begin { sub2 }
      ...
    end; { sub2 }

begin { principal }
  ...
end; { principal }
```

15. Asumiendo una estructura de bloques chatos y reglas de alcance dinámico, muestra un programa (utilizando una sintaxis similar a la de los ejercicios anteriores) en el cual diferentes secuencias de llamadas provoquen que las variables visibles en los procedimientos invocados difieran. Para ello, considere un bloque principal y al menos dos (2) procedimientos, con sus respectivas variables (no es necesario indicar el cuerpo de ejecución). Luego, muestre al menos tres secuencias de llamada distintas, y en cada caso indique qué variables son visibles durante la ejecución del último procedimiento activado y el nombre de la unidad en donde fue declarada.
16. Analice la utilidad de contar con una estructura de bloques anidados y reglas de alcance dinámico en un lenguaje de programación.

Casos de Estudio e Investigación

17. Identifique entidades que pueden aparecer en un programa de Java y no en uno de Pascal. Justifique su respuesta.
18. Considere las siguientes sentencias en Pascal (i) y Python (ii) respectivamente:

i. `x := y` ii. `x = y`

- a. Describa las ligaduras que se producen y qué atributos están vinculados.
- b. Describa la semántica de cada sentencia. ¿Qué relación encuentra entre esta descripción y las respuestas que dio en el inciso anterior?
19. Considerando la semántica de asignación de Python, ¿es necesario establecer o actualizar todas las ligaduras al realizar una asignación? ¿Qué política adopta Python? Ilustrar con un ejemplo.
20. En Go, una variable puede asignarse usando `=` o `:=`. Indique qué ligaduras se producen en cada caso.
21. Desarrolle un ejemplo en Pascal y contrástelo con otro en Python donde se muestre que la ligadura estática de tipos a las variables puede ayudar a la detección temprana de errores.
22. Investigue acerca de las diferencias entre `var`, `let` y `const` en Javascript. Describa cómo cambian las ligaduras de las variables al utilizar cada uno de estos constructores. Brinde ejemplos con los cuales puedan apreciarse sus diferencias. Explique por qué se recomienda el uso `let` y `const` por sobre el de `var`.
23. Considere el programa en Python que se muestra a la derecha, y que puede ejecutar en el siguiente enlace. Indique cuál es la salida de este programa y explique claramente la razón del resultado de cada “print”.

Luego responda:

- a. ¿Cuál es el propósito de la palabra reservada *global* en Python?
- b. ¿Qué sucedería si en *fun3* se elimina la declaración “global a”?

```
a = 8
def fun1():
    def fun2():
        def fun3():
            global a
            print(a)
            a=12
            fun3()
            print(a)
        a = 6
        fun2()
        print(a)
    fun1()
    print(a)
```

24. Perl es un lenguaje que permite al programador definir variables con alcance dinámico (pese a que por defecto usa reglas de alcance estático). Analice el siguiente código en Perl, detecte los puntos donde se utiliza alcance dinámico o alcance estático, y realice un breve análisis del impacto de contar con reglas de alcance dinámico a partir de los criterios de evaluación visto en la materia.

Nota: Ejecute el código usando la herramienta web tutorialspoint (enlace). Puede encontrar información del lenguaje en <https://www.perl.org/docs.html>.

25. Describa y ejemplifique alcance, tiempo de vida y visibilidad de las variables locales, de instancia y de clase en Java.
26. Considere las siguientes clases de Java. Indique qué elementos (no predefinidos) son visibles en el ambiente de referenciamiento del método `metA1` de la clase `A`, denotando qué calificadores son necesarios para su acceso.

```
class A {
    int a;
    String B;
    B metA2;
    public void metA1(boolean a) { ... }
    public void metA2() { ... }
}

class B {
    public static void metB1() { ... }
    private void metB2() { ... }
}
```

27. Ejecute los siguientes fragmentos de código escritos en Java, Javascript y Python, y explique la diferencia en los resultados obtenidos. Analice las ventajas y/o desventajas de cada enfoque. Nota: En este caso puede ejecutar los ejemplos (con alguna ligera modificación) en <https://onecompiler.com/>.

Java	JavaScript	Python
<pre>public class Main { static int y = f(); static int x = 1; public static void main (String[] args) { System.out.println(y); } public static int f() { return x*2; } }</pre>	<pre>var y = f(); var x = 1; function f(){ return x*2; } console.log(y)</pre>	<pre>y = f() x = 1 def f(): return x*2 print(y)</pre>